ASSIGNMENT 4 – TASK 4

EVENT-HANDLING FORENSICS

Course: Programming in Java | Course Outcome: CO4

Task value: 7 marks | Bloom's level: K4 – Analyse

1. Task objective

The drawing pad must draw a dot at each mouse click, display the total click count, and preserve all dots when the window is repainted. The investigation identifies the event-handler naming defect, demonstrates the repainting failure, repairs the program, and explains how compile-time checking prevents the same mistake.

2. Task 4.1 – Locate the defect

Incorrect method name: The original code declares `mouseclicked` with a lowercase "c". Java method names are case-sensitive. The MouseAdapter callback is `mouseClicked(MouseEvent e)`. Therefore, the method with the incorrect spelling does not override the callback, and the event system never invokes it. The click count remains 0 and the label is not updated.

Missing label update: Even if the callback name were corrected, the original handler only increments `count`; it does not call `status.setText(...)`. The label would continue to display "Clicks: 0".

Direct drawing is temporary: Calling `getGraphics()` and drawing directly is not persistent. AWT may repaint a window after it is uncovered, resized, minimised/restored, or otherwise invalidated. Directly drawn pixels are not retained as application state.

3. Task 4.2 – Construct a failing interaction

First correct the callback spelling so the click handler runs. Click several times and observe the dots. Then minimise the window and restore it (or cover it with another window and uncover it). The dots disappear because the drawing was performed only on the current screen graphics; the program did not save the click coordinates. During repainting, AWT calls paint again, but there is no stored drawing information to reproduce the dots.

4. Task 4.3 – Repair strategy

The corrected implementation stores every click as a `Point` in an `ArrayList`. In `mouseClicked`, it adds the new point, updates the label using the list size, and calls `repaint()`. The overridden `paint(Graphics g)` first calls `super.paint(g)`, then loops through the stored points and draws a 6 × 6 filled circle centred on each point using `fillOval(x - 3, y - 3, 6, 6)`.

This separates persistent state (the list of points) from the temporary screen pixels. Every repaint can reconstruct the drawing from the stored points.

5. Event sequence

Step	Event / method	Action
1	main()	Creates the DrawPad frame and makes it visible.
2	AWT event dispatch	Delivers a mouse click to the registered MouseListener.
3	mouseClicked(e)	Stores the click location as a Point.
4	status.setText(...)	Updates the visible click count.
5	repaint()	Requests an asynchronous repaint.
6	paint(g)	Redraws all saved points.
7	windowClosing(e)	Calls dispose() to close the window.

6. Task 4.4 – Prevention using @Override

The annotation `@Override` instructs the compiler to verify that the method really overrides a superclass or interface method. If the callback is misspelled as `mouseclicked`, compilation fails with a diagnostic similar to: `method does not override or implement a method from a supertype`. Without the annotation, the misspelled method can compile as an unrelated method and silently fail to receive mouse-click events.

7. Alternative concept – off-screen BufferedImage

Alternative: maintain a BufferedImage as an off-screen drawing surface. On a click, draw the new dot into the image using its Graphics2D object, then request repaint. In paint(), draw the saved image onto the window. The image acts as a persistent pixel buffer.

Measure	Stored Point list + redraw	Off-screen BufferedImage
Work per click	O(1) to append a point.	O(1) to draw one dot into the image.
Work per repaint	O(n), because n points are redrawn.	O(W × H) pixel-copy work to draw the image.
Storage	O(n) points.	O(W × H) pixels; typically about 4 bytes per pixel for an ARGB image.
Trade-off	Compact when there are few points; repaint cost grows with them.	Fast: never reconstruct the drawing from one image, but image memory depends

8. Test plan and evidence

Execute the corrected program and capture: (1) the window after several clicks, showing the updated count and dots; (2) the same window after minimising and restoring, showing the dots still present; and (3) the terminal output from compiling BrokenDrawPad.java, showing the @Override diagnostic. The screenshots must be genuine captures of the student's own execution and should be placed in the Task04 folder.

9. Conclusion

The original failure is caused by a case-sensitive callback-name mismatch and the use of transient screen drawing. Correctly overriding mouseClicked, storing click coordinates, updating the label, and redrawing the stored points in paint() produces a reliable AWT drawing pad. @Override exposes callback-name mistakes during compilation.